

Django Product Model

Exercises 1 & 2

- Django
- Models
- Migrations
- Django Admin
- Final Documentation

Complete practical documentation of the actual work: requirements, project setup, files modified, code, reasons for each change, migration troubleshooting, and browser verification screenshots.

IMPORTANT

Exercise 1 starts with a new `Product_Exercise` project. Exercise 2 continues in the same project and the same `products` app.

DOCUMENT	Exercises 1 & 2 — English Final Documentation
PROJECT	Product_Exercise
APP	products
CONTENTS	Requirements • Setup • Model code • Migrations • Browser verification

INSIDE THIS DOCUMENT	
01 Exercise 1 — Build the Product Model	04 Migration Problem and Solution
02 Exercise 1 — Browser Verification	05 Exercise 2 — Browser Verification
03 Exercise 2 — Continue the Same Project	06 Final Review — Exercises 1 & 2

EXERCISE 1

Build the Product Model

1 Exercise 1 Requirements

Exercise 1 started from scratch with a new Django project. The goal was to create the `products` app, build the `Product` model with the required fields, register it in Django Admin, and test it through the browser.

- Create a new Django project for the Product exercise.
- Create an app named `products`.
- Create the `Product` model with the required fields.
- Register `Product` in Django Admin.
- Install the required dependency and run migrations/checks.
- Create a superuser for Admin access.
- Test the `Product` model in the browser by adding a product named Keyboard.

2 New Project

We created a new project specifically for this exercise and named it:

```
project name
Product_Exercise
```

Inside it, we created the Django project `Product_project` and the app `products`.

```
structure
Product_Exercise/
├── venv/
├── Product_project/
├── products/
└── manage.py
```

3 settings.py – Connect the App

The first file we modified was `settings.py` inside `Product_project`. We added `products` to `INSTALLED_APPS`.

```
Product_project/settings.py
INSTALLED_APPS = [
    ...,
    'products',
]
```

WHY? Django needs to know that `products` is part of the project so it can discover its models and create migrations for them.

4 products/admin.py – Register Product

Next, we modified `admin.py` inside `products`. We imported `Product` and registered it in Django Admin.

```
products/admin.py
from django.contrib import admin
from .models import Product
admin.site.register(Product)
```

WHY? The exercise requires testing the `Product` model through Django Administration, so `Product` must be registered first.

5 products/models.py — Create the Product Model

Then we created the `Product` model in `products/models.py`. Each field below directly matches a requirement of Exercise 1.

```
products/models.py

from django.db import models

class Product(models.Model):
    name = models.CharField(max_length=120)

    description = models.TextField()

    price = models.DecimalField(
        max_digits=8,
        decimal_places=2
    )

    stock = models.PositiveIntegerField(default=0)

    is_active = models.BooleanField(default=True)

    available_from = models.DateField()

    created_at = models.DateTimeField(auto_now_add=True)

    product_image = models.ImageField(upload_to="products/")

    def __str__(self):
        return self.name
```

FIELD	WHAT WE USED	WHY
<code>name</code>	<code>CharField(max_length=120)</code>	Short product name, maximum 120 characters.
<code>description</code>	<code>TextField()</code>	Stores the product description.
<code>price</code>	<code>DecimalField(max_digits=8, decimal_places=2)</code>	Stores price with two decimal places.
<code>stock</code>	<code>PositiveIntegerField(default=0)</code>	Stock cannot be negative; default is 0.
<code>is_active</code>	<code>BooleanField(default=True)</code>	Stores active/inactive status; default is True.
<code>available_from</code>	<code>DateField()</code>	Stores the product availability date.
<code>created_at</code>	<code>DateTimeField(auto_now_add=True)</code>	Automatically records creation time.
<code>product_image</code>	<code>ImageField(upload_to='products/')</code>	Stores the product image reference under products/.

```
__str__()
```

We added it so the Admin displays the product name instead of a generic `Product object` representation.

6 Setup, Migrations, and Admin Access

We installed Pillow because `ImageField` requires it, then checked the project and prepared the database with migrations. Finally, we created a superuser for Django Admin.

Browser Verification

The following screenshots are the browser/Django Admin verification for Exercise 1 only. Code screenshots are intentionally not included.

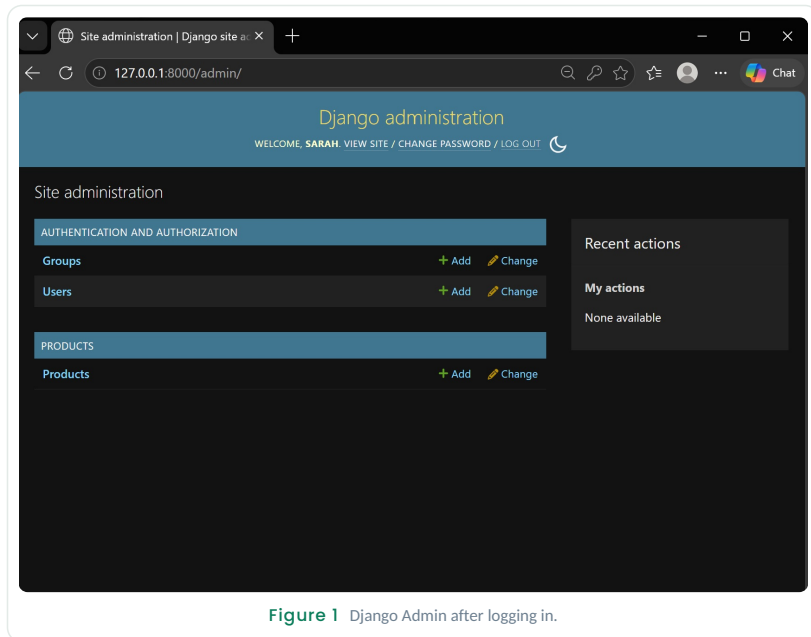


Figure 1 Django Admin after logging in.

Testing the Keyboard Product

We opened Add Product and started creating the required test product named Keyboard. During the first attempt, Django showed that some fields were required, so we completed the missing information and saved again.

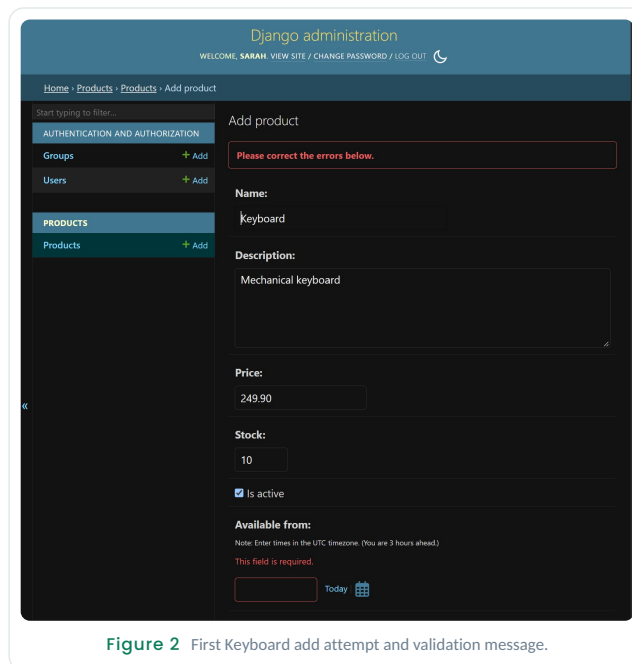


Figure 2 First Keyboard add attempt and validation message.

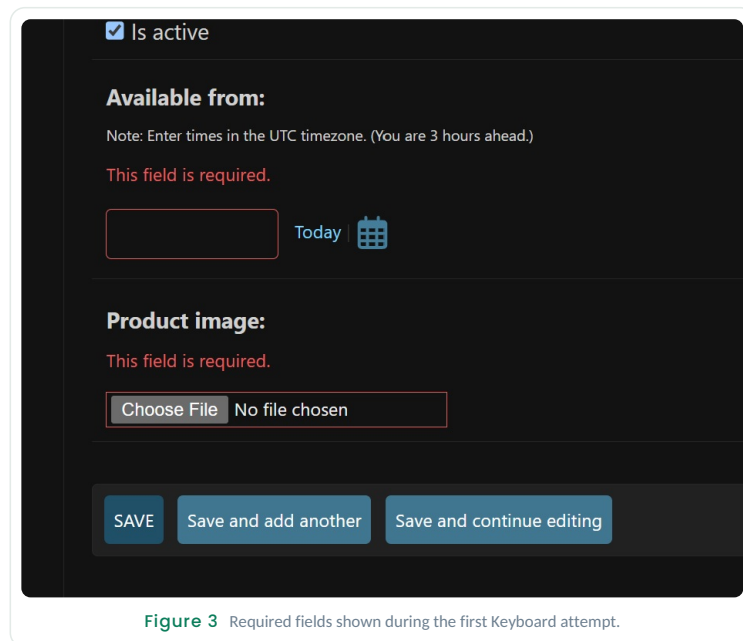


Figure 3 Required fields shown during the first Keyboard attempt.

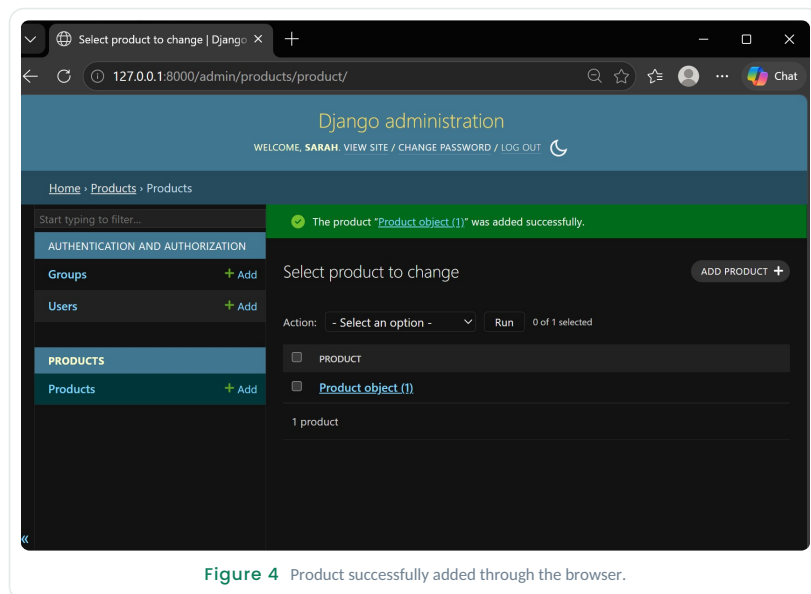


Figure 4 Product successfully added through the browser.

EXERCISE 1 COMPLETE

The new project, `products` app, `settings.py`, `admin.py`, `Product` model, Pillow, migrations, superuser, and Keyboard browser test were completed.

EXERCISE 2

Continue the Same Project

1 Exercise 2 Starts Here

Exercise 2 did **NOT** start with a new project. We continued directly from the same `Product_Exercise` project and the same `products` app created in Exercise 1.

```
same project

Product_Exercise
├── products
│   └── Product model
```

The new requirements were implemented by modifying the existing `Product` model in `products/models.py`.

2 The Eight Requirements

- 1 `description` must be optional without storing NULL.
- 2 `available_from` must be optional in validation and in the database.
- 3 Add `sku` with a maximum length of 30 characters and make it unique.
- 4 Add `category` using TextChoices: Laptop, Phone, Accessory.
- 5 Set Accessory as the default category and add `db_index=True` to `name`.
- 6 Do not define `id` manually; let Django create it automatically.
- 7 Explain the difference between `blank=True` and `null=True`.
- 8 Explain `product.pk` and the stored value for LAPTOP.

3 Changes in products/models.py

We modified the existing `Product` model to satisfy all eight requirements.

STEP 1 description

```
products/models.py

description = models.TextField(blank=True)
```

WHY? The description must be optional. `blank=True` allows an empty value during validation/forms without requiring NULL in the database.

STEP 2 available_from

```
products/models.py

available_from = models.DateField(
    null=True,
    blank=True
)
```

WHY? The field must be optional both during validation and at the database level, so we use `blank=True` and `null=True`.

STEP 3 sku

```
products/models.py

sku = models.CharField(
    max_length=30,
    unique=True
)
```

WHY? `max_length=30` enforces the required limit, while `unique=True` prevents duplicate SKUs.

STEP 4 category with TextChoices

```
products/models.py

class Category(models.TextChoices):
    LAPTOP = "laptop", "Laptop"
    PHONE = "phone", "Phone"
    ACCESSORY = "accessory", "Accessory"

    category = models.CharField(
        max_length=20,
        choices=Category.choices,
        default=Category.ACCESSORY
    )
```

WHY? The exercise requires a controlled list of categories. The first value is stored in the database; the second value is the human-readable label.

STEP 5 db_index and default

```
products/models.py

name = models.CharField(
    max_length=120,
    db_index=True
)

default=Category.ACCESSORY
```

WHY? The exercise explicitly requires an index on `name` and Accessory as the default category.

STEP 6 id

We did not define an `id` field manually. Django creates the primary key automatically when no primary key is specified.

STEP 7 blank=True vs null=True

	BLANK=TRUE	NULL=TRUE
Affects	Validation / Forms	Database
Meaning	Allows empty input	Allows NULL storage
Used here	<code>description + available_from</code>	<code>available_from</code>

STEP 8 product.pk and LAPTOP

```
products/models.py

LAPTOP = "laptop", "Laptop"

product.pk
```

The value stored for LAPTOP is `laptop`. Laptop is the display label. `product.pk` is the generic way to access the Product primary key.

4 Migration Problem and Solution

We already had an existing Product row from Exercise 1. When `sku` became required and unique, Django needed a value for that existing row while creating the migration.

```
terminal

python manage.py makemigrations

Select an option: 1
>>> "OLD-001"
```

OLD-001 was used as a one-off value for the existing row during the migration. It was not added as a permanent default in `models.py`.

```
terminal

python manage.py migrate
python manage.py check
```

VERIFICATION

The migration was applied successfully, and `python manage.py check` returned: System check identified no issues (0 silenced).

EXERCISE 2

Browser Verification

After applying the migration, we opened the same Django Admin to verify that the updated model was reflected in the browser.

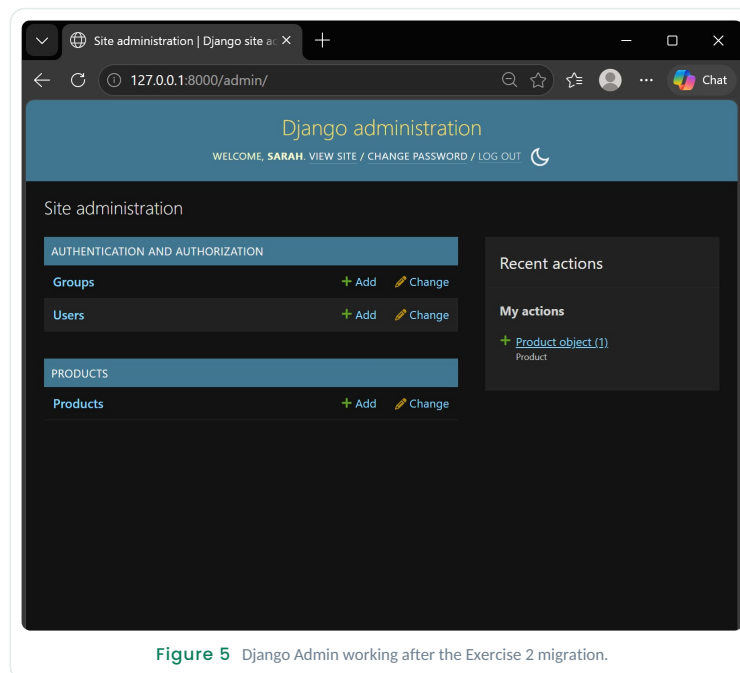


Figure 5 Django Admin working after the Exercise 2 migration.

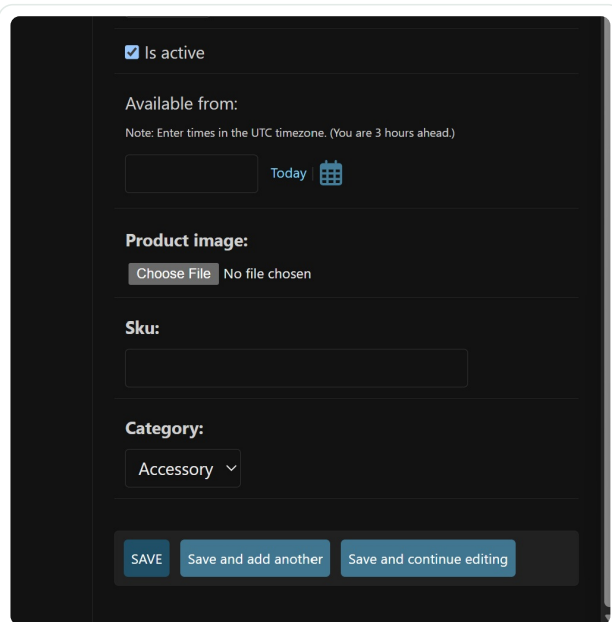


Figure 6 Add Product showing SKU and Category, with Accessory selected by default.

This browser check confirmed that SKU and Category appeared in the form and that Accessory was the default. The previous 'no such column: products_product.sku' error was also resolved after the migration.

EXERCISE 2 COMPLETE

All eight requirements were implemented, the database was synchronized with the model, and the result was verified in the browser.

FINAL REVIEW

Exercises 1 & 2

Exercise 1

- ✓ New Product_Exercise project
- ✓ products app created
- ✓ settings.py updated
- ✓ admin.py updated
- ✓ Product model created
- ✓ Pillow installed and migrations applied
- ✓ Superuser created
- ✓ Keyboard tested in the browser

Exercise 2

- ✓ Continued in the same Product_Exercise project
- ✓ Modified the same Product model
- ✓ Completed all eight requirements
- ✓ Created and applied the second migration
- ✓ Used OLD-001 as the one-off value for the existing row
- ✓ python manage.py check = 0 issues
- ✓ Verified SKU, Category, and Accessory in the browser

KEY POINT

Exercise 2 is a direct continuation of Exercise 1. No new project was created for Exercise 2.